

# LES FILTRES DE RECHERCHE



## LES EXPRESSIONS RÉGULIÈRES

Définition – Formule qui représente une chaîne de caractères.  
Composée de caractères et d'opérateurs.

Utilisation – On recherche alors non pas un mot ou une simple chaîne de caractères mais une suite de caractères qui correspondent au critères énoncés par la formule

– Certains opérateurs doivent être précédés d'un \ pour ne pas entrer en conflit avec le shell, ainsi : {, }, , (, ) et | seront écrits \{, \}, \, \{, \} et \|. C'est aussi le cas de l'espace



## LES EXPRESSIONS RÉGULIÈRES

### Rappel :

- \* N'importe quelle séquence de caractères
- ? N'importe quel caractère
- [] N'importe quel caractère choisi dans les caractères donnés entre crochets
- [^] n'importe quel caractère sauf ceux dans les caractères donnés entre crochets
- [-] n'importe quel caractère dans la plage de caractères donnés entre crochets

### Exemples :

\$ ls \*

\$ ls ?ateau

\$ ls \*ateau

\$ ls [gr]ate\*

\$ ls [ ^abr]ateau

\$ ls [a-c]\*



## LES MÉTA CARACTÈRES

Les expressions régulières sont aussi des suites de caractères permettant de faire des sélections. Elles sont utilisées avec certaines commandes comme *grep*.

Les différentes expressions régulières sont :

| Caractère spécial | Signification                                                           |
|-------------------|-------------------------------------------------------------------------|
|                   | Ou logique, l'expression située avant ou après doit apparaître.         |
| (...)             | Groupement de caractères.                                               |
| [...]             | Un caractère à cette position parmi ceux indiqués.                      |
| . (point)         | Un caractère quelconque.                                                |
| +                 | Répétition, le caractère placé avant doit apparaître au moins une fois. |
| *                 | Répétition, le caractère situé avant doit apparaître de zéro à n fois.  |
| ?                 | Le caractère situé avant doit apparaître une fois au plus.              |
| {n}               | Le caractère situé avant doit apparaître exactement n fois.             |
| {n,}              | Il apparaît n fois ou plus.                                             |
| {n,m}             | Il apparaît entre n et m fois.                                          |
| ^                 | En début de chaîne.                                                     |
| \$                | En fin de chaîne.                                                       |

## LES EXPRESSIONS RÉGULIÈRES

### Représentation :

- Un caractère compris entre 'a' et 'z' : [a-z]
- Un caractère compris entre '0' et '9' : [0-9]
- Le début d'une ligne : ^
- La fin d'une ligne : \$
- Un choix entre deux chaînes : \ |
- Et en combinant avec le choix : Caractère compris entre 'a' et 'z' ou 'A' et 'Z' : [a-zA-Z]
- Répétitions d'une occurrence
  - Exactement 2 répétitions de 'x' : x\{2\}
  - Entre 2 et 5 répétitions de 'x' : x\{2, 5\}
  - Au moins 2 répétitions de 'x' : x\{2, \}



## LES EXPRESSIONS RÉGULIÈRES

EXEMPLE :

- l'expression `[a-z][a-z] *` cherche les lignes contenant au minimum un caractère en minuscule. `[a-z]` caractère permis, `[a-z]*` recherche d'occurrence des lettres permises.
- L'expression `^[0-9]\{4\}` a pour signification, du début à la fin du fichier \$, recherche les nombres `[0-9]` de 4 chiffres `\{4\}`.
- L'expression `[a-z][a-z] *` cherche les lignes contenant au minimum un caractère en minuscule. `[a-z]` caractère permis, `[a-z]*` recherche d'occurrence des lettres permises.
- L'expression `^[0-9]\{4\}` a pour signification, du début à la fin du fichier \$, recherche les nombres `[0-9]` de 4 chiffres `\{4\}`.
- **Remarque :** Le caractère `^` n'a la signification de non qu'entre crochets.

Quelle est la différence entre `^[abc]` et `^[abc]` ?

## LES EXPRESSIONS RÉGULIÈRES

EXEMPLE :

- $\backslash[02-57]\backslash$  0, 2, 3, 4, 5 ou 7
- $\backslash[a-d5-8X-Z]\backslash$  a, b, c, d, 5, 6, 7, 8, X, Y ou Z
- $\backslash[0-5-]\backslash$  0, 1, 2, 3, 4, 5 ou -
- $\backslash[^0-9]\backslash$  pas un chiffre
- Un code permanent contient 4 majuscules suivies de 8 chiffres :  $[A-Z]\backslash\{4\}\backslash[0-9]\backslash\{8\}\backslash$
- Un code permanent contient 12 caractères lettres ou chiffres :  $[A-Z0-9]\backslash\{12\}\backslash$



## LES EXPRESSIONS RÉGULIÈRES

EXEMPLE PLUS AVANCÉ:

Repérer les lignes contenant une date qui est exprimée par  
JJ/MM/AA ou bien JJ/MM/AAAA

Jours 01 à 09 : 0[1-9]

Jours 10 à 29 : [1-2][0-9]

Jours 30 et 31: 3[0-1]

On obtient alors : `\(0[1-9]|[1-2][0-9]|3[0-1]\)`

Mois 01 à 09 : 0[1-9]

Mois 10 à 12 : 1[0-2]

on obtient alors : `\(0[1-9]|1[0-2]\)`

Une année contient simplement 2 ou 4 chiffres :

on obtient alors : `\([0-9]\{2\}|[0-9]\{4\}\)`

Concaténation des expressions formalisant les jours, mois et années avec  
un '/' entre elles : `\(0[1-9]|[1-2][0-9]|3[0-1]\)/\([0-9]|1[0-2]\)/\([0-9]\{2\}|[0-9]\{4\}\)`

## LA COMMANDE GREP

La commande grep permet de rechercher une chaîne de caractères dans un fichier. Les options sont les suivantes :

- v affiche les lignes ne contenant pas la chaîne
- c compte le nombre de lignes contenant la chaîne
- n chaque ligne contenant la chaîne est numérotée
- x ligne correspondant exactement à la chaîne
- l affiche le nom des fichiers qui contiennent la chaîne

Exemple avec le fichier carnet-adresse :

```
olivier:29:0298333242:Brest  
marcel:13:0466342233:Gardagnes  
myriam:30:0434214452:Nimes  
yvonne:92:013344433:Palaiseau
```

La commande :

```
$grep ^[g-n] carnet-adresse
```

Permet d'obtenir toutes les lignes commençant par les caractères compris entre g et n.

```
$grep Brest carnet-adresse
```

Permet d'obtenir les lignes contenant la chaîne de



## LA COMMANDE GREP

## ○ Les options de la commande grp

| Option                      | Signification                                                            |
|-----------------------------|--------------------------------------------------------------------------|
| -v                          | Afficher les lignes qui ne correspondent pas au critère                  |
| -c                          | Retourner le nombre de lignes qui correspond au critère                  |
| -i                          | Ne pas tenir en compte la casse                                          |
| -n                          | Chaque ligne trouvée est retournée avec son numéro de ligne              |
| -l                          | Lister uniquement les noms de fichiers qui correspondent au critère      |
| -a                          | Traiter un fichier binaire comme s'il s'agissait de texte                |
| -R, -r, -- <u>recursive</u> | Lire tous les fichiers à l'intérieur de chaque répertoire, récursivement |

## LA COMMANDE GREP

**Remarque** : Différence entre grep et find :

Find : sert à parcourir l'arborescence pour la recherche d'un nom de fichier

Grep : sert à chercher la chaîne de caractère qui lui est spécifiée dans les fichiers donnés en argument

### EXERCICE

Vérification très sommaire de la validité d'une adresse IP, « 4 décimaux de 3 chiffres qui sont séparés par des . ».



## LA COMMANDE GREP

## SOLUTION

```
echo $IP | grep '([0-9]{1,3}\.){3}([0-9]{1,3})'
```

Voici comment cette ligne est décomposée :

- ✓ `([0-9]{1,3}\.){3}` : `www.xxx.yyy.`
- ✓ `[0-9]` : un caractère entre 0 et 9
- ✓ `{1,3}` : répété entre une et trois fois, donc `x`, `xx` ou `xxx`
- ✓ `\.` : suivi d'un point
- ✓ `{3}` : le tout trois fois
- ✓ Puis `[09]{1,3}` : `.zzz`
- ✓ `[0-9]` : un caractère entre 0 et 9
- ✓ `{1,3}` : répété entre une et trois fois



## Exemples de Recherche avec find

## Opérateurs logiques

|    |                                         |
|----|-----------------------------------------|
| !  | Non logique                             |
| -a | Liaison par et logique de deux critères |
| -o | Liaison par ou logique de deux critères |

Exemple avec find

Les fichiers n'appartenant pas à l'utilisateur olivier:

```
find . ! -user olivier -print
```

Recherche des fichiers qui ont pour nom a.out et des fichiers se terminant par .c:

```
find . \(-name a.out -o -name "*.c"\) -print
```

Recherche des fichiers qui ont pour nom core et une taille supérieure à 1Mo.

```
find .\ (-name core -a -size +2000 \) -print
```

find utilisé avec d'autres commandes UNIX:

```
find . -type f -print| grep "test"
```



## Exemples de Recherche avec find

### ○ Remarque

Les opérateurs and et or n'ont pas la même priorité, donc, il faut positionner les parenthèses pour interpréter le ou logique avant le et logique. Les parenthèses étant à leur tour des caractères spéciaux, on doit les désactiver.



## LA COMMANDE FIND

### EXERCICES: FIND

- 1) Afficher tous les fichiers qui commencent par 'a' majuscule ou minuscule.
- 2) Afficher tous les fichiers qui se composent de 3 caractères dont le dernier est 'M'.
- 3) Sauvegarder tous les noms de fichiers se terminant soit par a, b ou c dans un fichier F1.
- 4) Créer les fichiers suivants dans un répertoire : a1, a2, a3.....a9 et afficher les 5 premiers
- 5) Afficher tous les noms de fichiers répertoires commençant par une lettre majuscule.
- 6) Afficher tous les noms de fichiers de l'utilisateur « webmaster1 » qui appartient au groupe « webos ».
- 7) Afficher tous les fichiers normaux ayant pour taille plus de 150 Ko.
- 8) Afficher tous les fichiers dont les dernières modifications remontent à moins de trois jours.
- 9) Afficher tous les répertoires ayant les autorisations d'accès 'rwxr-xr-x'.
- 10) Supprimer tous les fichiers auxquels vous n'avez pas accédés depuis plus de deux semaines.
- 11) Afficher tous les fichiers répertoires ou les fichiers dont le nom se termine par le caractère 'a'.
- 12) Afficher tous les fichiers qui n'appartiennent pas à l'utilisateur « userxgl »

## LA COMMANDE FIND

## SOLUTION: FIND

- `$find / -name "[a-A] *" -print`
- `$find / -name "??M" -print`
- `$find / -name "[a b c] " > F1`
- `$touch a1 a2 a3 a4 a5 a6 a7 a8 a9 | find -name "a[1-5]"`
- `$find / -type d -name "[A-Z] *"`
- `$find / -user "webmasters1" -a -group "webos"`
- `$find /-type f -size "+150K"`
- `$find /-type f -mtime -3`
- `$find /-type d -perm 755`
- `$find /-type f -atime +15 -exec rm`
- `$find /(-type d -o type f) -name "*a"`
- `$find ! -user "user xg"`



## EXERCICES: CUT, GREP, TAIL, HEAD, TR

A partir du fichier etc/passwd

- ❑ Affichez uniquement la ligne concernant l'utilisateur root
- ❑ Affichez uniquement les login de tous les utilisateurs triés (1<sup>er</sup> champ).
- ❑ Affichez les correspondances login (1) numéro d'utilisateur (3), nom réel(5)
- ❑ affichez le nom des utilisateurs du groupe(4) 1000 et leur groupe.
- ❑ Combien y'en a-t-il ?
- ❑ Quels sont les fichiers de /etc/ contenant des nombres de plus de trois chiffres
- ❑ Afficher la 7<sup>ième</sup> ligne de ce fichier
- ❑ Afficher le fichier /etc/passwd en remplaçant les caractères / par des X.



### EXERCICE

- Lister le contenu du répertoire /dev.
- Lister le contenu du fichier /etc/passwd.
- Afficher par ordre alphabétique les utilisateurs définis dans le fichier /etc/passwd.
- Rechercher tous les fichiers du répertoire /etc contenant la chaîne de caractères "root".
- Rechercher la localisation du fichier "stdio.h" dans le système de fichier de votre installation.
- Utiliser les commandes de comparaison de fichiers de Linux pour tester le contenu d'un même fichier texte en version Unix et Dos.
- Combien de lignes, de mots et de caractères comportent les fichiers "montexte.unix", "montexte.dos" et "montexte.mac"?



# LES ALIAS

Il permet de donner un synonyme à une commande ou à un groupement de commandes.

Cela permet de gagner du temps et de simplifier les commandes.

On peut mettre un certain nombre d'alias dans le fichier .profile

La syntaxe est la suivante : `alias nom_alias = 'commande '`

Exemple :

*\$ alias l='ls-al | more' # est une commande très #pratique.*

*\$alias goappli='cd /home/compte\_appli/appli'*

Pour obtenir la liste des alias définis : `alias`